FINÁLNÍ PROJEKT
č. 2



Autor: Oksana Bouškova
Datum: 9. 10. 2026

OBSAH

ZADÁNÍ 3
TESTOVACÍ SCÉNÁŘE 4
EXEKUCE TESTŮ 12
BUG REPORT 15

ZADÁNÍ

Cílem finálního projektu je otestovat funkčnost aplikace, která slouží k manipulaci s daty o studentech. Aplikace má rozhraní REST-API, které umožňuje vytvoření, smazání a získání dat..

Přístupové údaje:

Databáze	**Default scheme**: students **Host**: test-app.engeto.cz **Port**: 5433
REST-API REST-API dokumentace	https://test-app.engeto.cz/students https://test-app.engeto.cz/docs

Poznámky:

Nezapomeňte, že v IT se data musí někde uložit a poté získat. Proto ověřte, že data jsou správně uložena a získávána z databáze.
Nezapomeňte do testovacích scénářů uvést testovací data, očekávaný výsledek včetně těla odpovědi a stavových kódů.

TESTOVACÍ SCÉNÁŘE

Na základě uvedených testovacích scénářů jsem ověřila funkčnost aplikace.
Nástroje: Postman (volání REST API, výstupy z Postman Console) a DBeaver (ověření dat v databázi PostgreSQL). Požadavky i odpovědi jsou uvedeny jako text; z výpisů jsou odstraněny nepodstatné hlavičky a token.

Příprava: přihlášení přes POST https://test-app.engeto.cz/login (login a heslo dle zadání), získaný accessToken se posílá v hlavičce Authorization: Bearer <token>. Testovací studenti mají e-mail s předponou oksana.test2. (kromě TS09, který ověřuje neplatný formát e-mailu), aby šlo snadno rozlišit vlastní data. Token ani přihlašovací údaje nejsou v dokumentu uvedeny.

Typ scénáře: P = pozitivní, N = negativní, H = hraniční. Scénáře TS23–TS27 byly doplněny po kontrole dokumentace API.

Autorizace

TS01 – Přihlášení se špatným heslem *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/login
Content-Type: application/json

{
  "login": "<login>",
  "password": "spatneheslo"
}
```

Očekávaný výsledek: 401 Unauthorized, token se nevrátí. Tělo: {"detail": "Invalid credentials", ...}

TS02 – Přihlášení s heslem kratším než 6 znaků *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/login
Content-Type: application/json

{
  "login": "<login>",
  "password": "123"
}
```

Očekávaný výsledek: 422 Unprocessable Entity, validační chyba pole password (min. 6 znaků).

TS23 – Přihlášení bez pole password *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/login
Content-Type: application/json

{
  "login": "<login>"
}
```

Očekávaný výsledek: 422 Unprocessable Entity, chybí povinné pole password.

TS03 – Požadavek na /students bez tokenu *(negativní)*

Požadavek:

```
GET https://test-app.engeto.cz/students/1
(bez hlavičky Authorization)
```

Očekávaný výsledek: 401 Unauthorized (dokumentace: „Missing or invalid bearer token“).

TS04 – Požadavek s neplatným tokenem *(negativní)*

Požadavek:

```
GET https://test-app.engeto.cz/students/1
Authorization: Bearer neplatnytoken123
```

Očekávaný výsledek: 401 Unauthorized.

TS24 – Založení studenta bez tokenu *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Content-Type: application/json
(bez hlavičky Authorization)

{
  "firstName": "X",
  "email": "oksana.test2.notoken@example.com"
}
```

Očekávaný výsledek: 401 Unauthorized, záznam nevznikne.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.notoken@example.com';
-- očekávám 0 řádků
```

POST – založení studenta

TS05 – Založení studenta s platnými daty *(pozitivní)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Jana",
  "lastName": "Testová",
  "email": "oksana.test2.jana@example.com",
  "isEuCitizen": true,
  "zipCode": "25001",
  "numberOfFailedStudies": 0
}
```

Očekávaný výsledek: 201 Created, v těle objekt studenta s přiděleným id a stejnými hodnotami jako v požadavku. V DB 1 řádek se shodnými hodnotami.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.jana@example.com';
```

TS06 – Založení studenta s již existujícím e-mailem *(negativní)*

Předpoklad: TS05 proveden.

Požadavek:

```
POST https://test-app.engeto.cz/students/
(stejný požadavek jako v TS05)
```

Očekávaný výsledek: 400 Bad Request (dokumentace: „Student with the same e-mail already exists“), nový záznam nevznikne.

Ověření v DB:

```
SELECT COUNT(*) FROM students WHERE email = 'oksana.test2.jana@example.com';
-- očekávám 1
```

TS07 – Záporný počet neúspěšných studií *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Petr",
  "lastName": "Test",
  "email": "oksana.test2.petr@example.com",
  "numberOfFailedStudies": -1
}
```

Očekávaný výsledek: 422, numberOfFailedStudies musí být ≥ 0. Záznam se neuloží.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.petr@example.com';
-- očekávám 0 řádků
```

TS08 – Neplatné datové typy *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Eva",
  "email": "oksana.test2.eva@example.com",
  "isEuCitizen": "ano",
  "numberOfFailedStudies": "dva"
}
```

Očekávaný výsledek: 422, validační chyba u isEuCitizen i numberOfFailedStudies. Záznam se neuloží.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.eva@example.com';
-- očekávám 0 řádků
```

TS09 – Neplatný formát e-mailu *(negativní)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Karel",
  "lastName": "Test",
  "email": "oksana2-neni-email"
}
```

Očekávaný výsledek: 422, hodnota není platný e-mail. Záznam se neuloží.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana2-neni-email';
-- očekávám 0 řádků
```

TS10 – Prázdné tělo požadavku *(hraniční)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{}
```

Očekávaný výsledek: 422, student bez jména a e-mailu nedává smysl. Záznam se neuloží.

Ověření v DB:

```
SELECT * FROM students WHERE id = <id z odpovědi>;
-- záznam nesmí existovat
```

TS11 – firstName delší než 255 znaků *(hraniční)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "aaaa…a" (256 znaků),
  "email": "oksana.test2.dlouhe@example.com"
}
```

Očekávaný výsledek: 422, firstName může mít max. 255 znaků.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.dlouhe@example.com';
-- očekávám 0 řádků
```

TS25 – zipCode delší než 50 znaků *(hraniční)*

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Zip",
  "email": "oksana.test2.zip@example.com",
  "zipCode": "1111…1" (51 znaků)
}
```

Očekávaný výsledek: 422, zipCode může mít max. 50 znaků.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.zip@example.com';
-- očekávám 0 řádků
```

TS26 – Duplicitní e-mail zapsaný velkými písmeny *(hraniční)*

Předpoklad: TS05 proveden.

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Jana",
  "lastName": "Velka",
  "email": "OKSANA.TEST2.JANA@EXAMPLE.COM"
}
```

Očekávaný výsledek: 400, e-mailová adresa nerozlišuje velikost písmen, jde o stejný e-mail jako v TS05.

Ověření v DB:

```
SELECT id, email FROM students WHERE lower(email) = 'oksana.test2.jana@example.com';
-- očekávám 1 řádek
```

GET – zobrazení studenta

TS12 – Zobrazení existujícího studenta *(pozitivní)*

Předpoklad: Student 357 z TS05.

Požadavek:

```
GET https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
```

Očekávaný výsledek: 200 OK, data v odpovědi odpovídají záznamu v DB.

Ověření v DB:

```
SELECT * FROM students WHERE id = 357;
```

TS13 – Zobrazení neexistujícího studenta *(negativní)*

Požadavek:

```
GET https://test-app.engeto.cz/students/999999999
Authorization: Bearer <token>
```

Očekávaný výsledek: 404 Not Found.

Ověření v DB:

```
SELECT * FROM students WHERE id = 999999999;
-- očekávám 0 řádků
```

TS14 – ID v neplatném formátu *(negativní)*

Požadavek:

```
GET https://test-app.engeto.cz/students/abc
Authorization: Bearer <token>
```

Očekávaný výsledek: 422, student_id musí být celé číslo.

PUT – úprava studenta

TS15 – Úprava existujícího studenta *(pozitivní)*

Předpoklad: Student 357 z TS05.

Požadavek:

```
PUT https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Jana",
  "lastName": "Upravená",
  "email": "oksana.test2.jana@example.com",
  "isEuCitizen": false,
  "zipCode": "25001",
  "numberOfFailedStudies": 1
}
```

Očekávaný výsledek: 200 OK, v odpovědi i v DB upravená data (Upravená, false, 1, zip 25001).

Ověření v DB:

```
SELECT * FROM students WHERE id = 357;
```

TS16 – Úprava neexistujícího studenta *(negativní)*

Požadavek:

```
PUT https://test-app.engeto.cz/students/999999999
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Jana",
  "lastName": "Upravená",
  "email": "oksana.test2.duch@example.com"
}
```

Očekávaný výsledek: 404 Not Found, v DB nevznikne nový záznam.

Ověření v DB:

```
SELECT * FROM students WHERE email = 'oksana.test2.duch@example.com';
-- očekávám 0 řádků
```

TS17 – Úprava s neplatnými daty *(negativní)*

Předpoklad: TS15 proveden (numberOfFailedStudies = 1).

Požadavek:

```
PUT https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
Content-Type: application/json

{
  "numberOfFailedStudies": -5
}
```

Očekávaný výsledek: 422, data v DB se nezmění (number_of_failed_studies stále 1).

Ověření v DB:

```
SELECT number_of_failed_studies FROM students WHERE id = 357;
```

TS18 – Změna e-mailu na e-mail jiného studenta *(negativní)*

Předpoklad: Existuje student 358 s e-mailem oksana2-neni-email (z TS09).

Požadavek:

```
PUT https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
Content-Type: application/json

{
  "email": "oksana2-neni-email"
}
```

Očekávaný výsledek: 400, e-mail musí být jedinečný (stejně jako u POST v TS06). Data v DB se nezmění.

Ověření v DB:

```
SELECT * FROM students WHERE id IN (357, 358);
```

TS19 – Částečná úprava – jen jméno *(hraniční)*

Požadavek:

```
PUT https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "Janička"
}
```

Očekávaný výsledek: 200, změní se jen first_name, ostatní pole zůstanou beze změny (dokumentace: „partial payload“).

Ověření v DB:

```
SELECT * FROM students WHERE id = 357;
-- porovnat se stavem před požadavkem
```

DELETE – smazání studenta

TS20 – Smazání existujícího studenta *(pozitivní)*

Předpoklad: Student 357.

Požadavek:

```
DELETE https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
```

Očekávaný výsledek: 200 OK (dle dokumentace), záznam v DB zmizí.

Ověření v DB:

```
SELECT * FROM students WHERE id = 357;
-- očekávám 0 řádků
```

TS21 – Opakované smazání téhož studenta *(hraniční)*

Předpoklad: TS20 proveden.

Požadavek:

```
DELETE https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
```

Očekávaný výsledek: 404 Not Found.

TS22 – Zobrazení smazaného studenta *(hraniční)*

Předpoklad: TS20 proveden.

Požadavek:

```
GET https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
```

Očekávaný výsledek: 404 Not Found.

TS27 – Smazání neexistujícího studenta *(negativní)*

Požadavek:

```
DELETE https://test-app.engeto.cz/students/999999999
Authorization: Bearer <token>
```

Očekávaný výsledek: 404 Not Found.

EXEKUCE TESTŮ

Testovací scénáře jsem provedla, přikládám výsledky testů.
Testy provedeny 9. 10. 2026 v Postmanu na prostředí https://test-app.engeto.cz. Výsledek každého scénáře byl ověřen dotazem do databáze students.

Souhrn výsledků

Oblast	Počet TS	Pass	Fail
Autorizace	6	4	2
POST	9	5	4
GET	3	2	1
PUT	5	2	3
DELETE	4	0	4
Celkem	**27**	**13**	**14**

Výsledky jednotlivých scénářů

TS	Skutečný výsledek (stavový kód, tělo odpovědi, DB)	Stav	Defekt
Autorizace			
TS01	`401 Unauthorized {"detail":"Invalid credentials","code":"INVALID_CREDENTIALS"}`	**Pass**	
TS02	`422 Unprocessable Entity {"detail":"body -> password: String should have at least 6 characters","code":"VALIDATION_ERROR"}`	**Pass**	
TS23	`422 Unprocessable Entity {"detail":"body -> password: Field required","code":"VALIDATION_ERROR"}`	**Pass**	
TS03	`403 Forbidden {"detail":"Not authenticated","code":"FORBIDDEN"}`	**Fail**	BUG-01
TS04	`401 Unauthorized {"detail":"Invalid token","code":"INVALID_TOKEN"}`	**Pass**	
TS24	`403 Forbidden {"detail":"Not authenticated","code":"FORBIDDEN"}; v DB 0 řádků`	**Fail**	BUG-01
POST – založení studenta			
TS05	`201 Created {"id":357,"firstName":"Jana","lastName":"TESTOVÁ","email":"oksana.test2.jana@example.com","isEuCitizen":true,"numberOfFailedStudies":0}; v DB last_name = TESTOVÁ, zip_code prázdné`	**Fail**	BUG-02, BUG-03
TS06	`400 Bad Request {"detail":"Student with the same e-mail already exists","code":"STUDENT_EMAIL_EXISTS"}; v DB 1 záznam (357)`	**Pass**	
TS07	`422 {"detail":"body -> numberOfFailedStudies: Input should be greater than or equal to 0"}; v DB 0 řádků`	**Pass**	
TS08	`422 – chyba u isEuCitizen (valid boolean) i numberOfFailedStudies (valid integer); v DB 0 řádků`	**Pass**	

TS	Skutečný výsledek (stavový kód, tělo odpovědi, DB)	Stav	Defekt
TS09	`201 Created {"id":358,"firstName":"Karel","lastName":"TEST","email":"oksana2-neni-email",...}; záznam 358 v DB`	**Fail**	BUG-04
TS10	`201 Created {"id":359,"firstName":null,"lastName":null,"email":null,"isEuCitizen":false,"numberOfFailedStudies":0}; prázdný záznam 359 v DB`	**Fail**	BUG-06
TS11	`422 {"detail":"body -> firstName: String should have at most 255 characters"}; v DB 0 řádků`	**Pass**	
TS25	`422 {"detail":"body -> zipCode: String should have at most 50 characters"}; v DB 0 řádků`	**Pass**	
TS26	`201 Created {"id":360,"firstName":"Jana","lastName":"VELKA","email":"OKSANA.TEST2.JANA@EXAMPLE.COM",...}; v DB 2 záznamy se stejnou adresou (357, 360)`	**Fail**	BUG-07
GET – zobrazení studenta			
TS12	`200 OK {"id":357,"firstName":"Jana","lastName":"TESTOVÁ","email":"oksana.test2.jana@example.com","isEuCitizen":true,"numberOfFailedStudies":0}; shoda s DB`	**Pass**	
TS13	`500 Internal Server Error {"detail":"Chyba, nelze najit studenta","code":"NOT_FOUND"}; v DB 0 řádků`	**Fail**	BUG-05
TS14	`422 {"detail":"path -> student_id: Input should be a valid integer, unable to parse string as an integer"}`	**Pass**	
PUT – úprava studenta			
TS15	`200 OK {"id":357,"firstName":"Jana","lastName":"Upravená","isEuCitizen":false,"numberOfFailedStudies":1,...}; v DB změny uložené, zip_code prázdné`	**Fail**	BUG-03
TS16	`404 Not Found {"detail":"Chyba, nelze zmenit data neexistujuceho studenta","code":"NOT_FOUND"}; v DB 0 řádků`	**Pass**	
TS17	`422 {"detail":"body -> numberOfFailedStudies: Input should be greater than or equal to 0"}; v DB stále 1`	**Pass**	
TS18	`200 OK {"id":357,...,"email":"oksana2-neni-email","isEuCitizen":false,...}; v DB 357 i 358 se stejným e-mailem`	**Fail**	BUG-08
TS19	`200 OK {"id":357,"firstName":"Janička",...,"isEuCitizen":true,...}; jméno změněno, ale is_eu_citizen se v DB přepnul z false na true`	**Fail**	BUG-09
DELETE – smazání studenta			
TS20	`200 OK {"message":"Student deleted successfully"}; záznam 357 v DB zůstal`	**Fail**	BUG-10
TS21	`200 OK {"message":"Student deleted successfully"}`	**Fail**	BUG-10
TS22	`200 OK {"id":357,"firstName":"Janička","lastName":"Upravená",...}`	**Fail**	BUG-10
TS27	`500 Internal Server Error {"detail":"Chyba, nelze najit studenta","code":"NOT_FOUND"}`	**Fail**	BUG-05

Ověření dat v DB po testech

Stav vlastních testovacích záznamů po pokusu o jejich smazání (DELETE vrátil u všech 200):

```
SELECT id, first_name, last_name, email FROM students
WHERE id BETWEEN 357 AND 360 ORDER BY id;

id |first_name|last_name|email                        |
---+----------+---------+-----------------------------+
357|Janička   |Upravená |oksana2-neni-email           |
358|Karel     |TEST     |oksana2-neni-email           |
359|          |         |                             |
360|Jana      |VELKA    |OKSANA.TEST2.JANA@EXAMPLE.COM|
```

Vlastní testovací záznamy 357–360 nebylo možné odstranit, protože DELETE data nemaže (BUG-10). Záznam 360 byl poté ještě použit pro opakované ověření BUG-09 (first_name nyní „Pokus4“). Cizí záznamy nebyly mazány ani upravovány.

Další postřehy

– Pole, která klient neposlal, dostanou výchozí hodnoty (isEuCitizen = false, numberOfFailedStudies = 0) místo null. Dokumentace toto chování neuvádí.

– Validace délek řetězců, datových typů a záporných čísel funguje správně u POST i PUT.

– Chybové hlášky kombinují angličtinu a češtinu bez diakritiky („Chyba, nelze najit studenta“).

– POST na /students (bez koncového lomítka) vrací 307 Temporary Redirect na /students/.

BUG REPORT

Na základě provedených scénářů jsem objevila uvedené chyby aplikace.

bug	probability	severity	test case number	mitigace
BUG-10: DELETE hlásí úspěch, ale záznam nesmaže	Vysoká (vždy)	Vysoká	TS20, TS21, TS22	Opravit mazání v DB; vracet 200/204 jen po skutečném smazání a 404 pro neexistující záznam.
BUG-09: PUT bez pole isEuCitizen tuto hodnotu obrátí	Střední (8 z 10 pokusů)	Vysoká	TS19	Při částečné úpravě měnit jen pole obsažená v požadavku.
BUG-08: PUT dovolí nastavit e-mail jiného studenta	Vysoká (vždy)	Vysoká	TS18	Kontrolovat jedinečnost e-mailu i při PUT a vracet 400 jako u POST; unikátní index v DB.
BUG-05: Neexistující ID vrací 500 místo 404 (GET, DELETE)	Vysoká (vždy)	Vysoká	TS13, TS27	Vracet 404 Not Found, stejně jako PUT (TS16).
BUG-01: Požadavek bez tokenu vrací 403 místo 401	Vysoká (vždy)	Střední	TS03, TS24	Při chybějící hlavičce Authorization vracet 401 Unauthorized.
BUG-02: POST ukládá příjmení velkými písmeny	Vysoká (vždy)	Střední	TS05	Ukládat příjmení beze změny, stejně jako PUT.
BUG-03: zipCode se neukládá (POST ani PUT)	Vysoká (vždy)	Střední	TS05, TS15	Ukládat zipCode do sloupce zip_code a vracet ho v odpovědi (doplnit do schématu Student).
BUG-04: POST nekontroluje formát e-mailu	Vysoká (vždy)	Střední	TS09	Validovat formát e-mailu (např. typ EmailStr) a vracet 422.
BUG-06: POST založí studenta bez jména a e-mailu	Vysoká (vždy)	Střední	TS10	Označit firstName, lastName a email jako povinné a doplnit to do dokumentace.
BUG-07: Jedinečnost e-mailu rozlišuje velká a malá písmena	Vysoká (vždy)	Nízká	TS26	Porovnávat e-maily bez ohledu na velikost písmen (ukládat v malých písmenech).

Detail defektů

Prostředí pro všechny defekty: https://test-app.engeto.cz, Postman + DBeaver, 9. 10. 2026. Token je v požadavcích nahrazen textem <token>.

BUG-10 – DELETE hlásí úspěch, ale záznam nesmaže

Kroky k reprodukci: 1. Smazat existujícího studenta 357. 2. Ověřit záznam v DB. 3. Zopakovat DELETE a zavolat GET.

Požadavek:

```
DELETE https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
```

Očekávaný výsledek: 200 nebo 204, záznam v DB zmizí; opakovaný DELETE i GET vrátí 404.

Skutečný výsledek: 200 {"message":"Student deleted successfully"}, záznam v DB zůstal. Opakovaný DELETE vrací znovu 200, GET vrací 200 s daty. Stejně se chovaly všechny 4 testovací záznamy (357–360).

Důkaz z DB:

```
SELECT id, first_name, last_name, email FROM students WHERE id = 357;
id |first_name|last_name|email             |
357|Janička   |Upravená |oksana2-neni-email|
```

BUG-09 – PUT bez pole isEuCitizen tuto hodnotu obrátí

Kroky k reprodukci: 1. Student 357 má v DB is_eu_citizen = false. 2. Poslat PUT, který pole isEuCitizen neobsahuje. 3. Ověřit DB.

Požadavek:

```
PUT https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
Content-Type: application/json

{ "firstName": "Janička" }
```

Očekávaný výsledek: 200, změní se jen first_name, is_eu_citizen zůstane false.

Skutečný výsledek: Odpověď i DB obsahují is_eu_citizen = true. Chyba se neprojevuje vždy: v TS18 (jen e-mail) hodnota zůstala false, v TS19 se přepnula. Opakovaný test na studentovi 360 se stejným typem požadavku (jen firstName): hodnota se přepnula ve 3 ze 4 pokusů. V předběžném běhu 7. 10. 2026 se přepnula ve 4 ze 4 pokusů. Celkem 8 z 10 PUT požadavků bez isEuCitizen hodnotu změnilo.

Důkaz z DB:

```
SELECT id, first_name, is_eu_citizen FROM students WHERE id = 357;
357|Jana      |false   (po TS18, před TS19)
357|Janička   |true    (po TS19)

-- opakování: PUT /students/360 { "firstName": "PokusN" }, po každém pokusu:
SELECT id, first_name, is_eu_citizen FROM students WHERE id = 360;
360|Janička   |false   (výchozí stav)
360|Pokus1    |false   (pokus 1 – beze změny)
360|Pokus2    |true    (pokus 2 – přepnuto)
360|Pokus3    |false   (pokus 3 – přepnuto)
360|Pokus4    |true    (pokus 4 – přepnuto)
```

BUG-08 – PUT dovolí nastavit e-mail jiného studenta

Kroky k reprodukci: 1. Existují studenti 357 a 358. 2. Změnit e-mail studenta 357 na e-mail studenta 358.

Požadavek:

```
PUT https://test-app.engeto.cz/students/357
Authorization: Bearer <token>
Content-Type: application/json

{ "email": "oksana2-neni-email" }
```

Očekávaný výsledek: 400, stejně jako POST s existujícím e-mailem (TS06).

Skutečný výsledek: 200, v DB jsou dva studenti se stejným e-mailem.

Důkaz z DB:

```
SELECT * FROM students WHERE id IN (357, 358);
id |first_name|last_name|email             |
358|Karel     |TEST     |oksana2-neni-email|
357|Jana      |Upravená |oksana2-neni-email|
```

BUG-05 – Neexistující ID vrací 500 místo 404 (GET, DELETE)

Kroky k reprodukci: 1. Zavolat GET a DELETE na ID, které v DB neexistuje.

Požadavek:

```
GET https://test-app.engeto.cz/students/999999999
Authorization: Bearer <token>

DELETE https://test-app.engeto.cz/students/999999999
Authorization: Bearer <token>
```

Očekávaný výsledek: 404 Not Found (dle dokumentace). PUT na stejné ID správně vrací 404.

Skutečný výsledek: 500 Internal Server Error {"detail":"Chyba, nelze najit studenta","code":"NOT_FOUND"}

Důkaz z DB:

```
SELECT * FROM students WHERE id = 999999999;
(0 rows)
```

BUG-01 – Požadavek bez tokenu vrací 403 místo 401

Kroky k reprodukci: 1. Zavolat endpoint /students bez hlavičky Authorization.

Požadavek:

```
GET https://test-app.engeto.cz/students/1
(bez hlavičky Authorization)
```

Očekávaný výsledek: 401 Unauthorized (dokumentace: „Missing or invalid bearer token“).

Skutečný výsledek: 403 Forbidden {"detail":"Not authenticated","code":"FORBIDDEN"}. Neplatný token přitom správně vrací 401 (TS04).

BUG-02 – POST ukládá příjmení velkými písmeny

Kroky k reprodukci: 1. Založit studenta s příjmením „Testová“. 2. Ověřit odpověď a DB.

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{ "firstName": "Jana", "lastName": "Testová",
  "email": "oksana.test2.jana@example.com", "isEuCitizen": true,
  "zipCode": "25001", "numberOfFailedStudies": 0 }
```

Očekávaný výsledek: Příjmení uložené tak, jak bylo odesláno: Testová.

Skutečný výsledek: Odpověď i DB obsahují TESTOVÁ (stejně „TEST“ u TS09 a „VELKA“ u TS26). PUT příjmení neupravuje (TS15 uložil „Upravená“).

Důkaz z DB:

```
SELECT id, first_name, last_name, zip_code FROM students
WHERE email = 'oksana.test2.jana@example.com';
id |first_name|last_name|zip_code|
357|Jana      |TESTOVÁ  |        |
```

BUG-03 – zipCode se neukládá (POST ani PUT)

Kroky k reprodukci: 1. Založit (TS05) nebo upravit (TS15) studenta se zipCode „25001“. 2. Ověřit DB.

Požadavek:

```
(požadavek TS05 – viz BUG-02, případně TS15)
```

Očekávaný výsledek: Hodnota 25001 ve sloupci zip_code.

Skutečný výsledek: API pole přijme a odpoví 201/200, ale zip_code v DB zůstane prázdný. Pole chybí i v odpovědi API.

Důkaz z DB:

```
SELECT id, last_name, is_eu_citizen, zip_code, number_of_failed_studies
FROM students WHERE id = 357;   -- po TS15
id |last_name|is_eu_citizen|zip_code|number_of_failed_studies|
357|Upravená |false        |        |                       1|
```

BUG-04 – POST nekontroluje formát e-mailu

Kroky k reprodukci: 1. Založit studenta s e-mailem bez zavináče a domény.

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{ "firstName": "Karel", "lastName": "Test", "email": "oksana2-neni-email" }
```

Očekávaný výsledek: 422, hodnota není e-mail.

Skutečný výsledek: 201 Created, vznikl záznam 358.

BUG-06 – POST založí studenta bez jména a e-mailu

Kroky k reprodukci: 1. Poslat POST s prázdným JSON objektem.

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{}
```

Očekávaný výsledek: 422, student bez jména a e-mailu nedává smysl.

Skutečný výsledek: 201 Created, prázdný záznam 359. Dokumentace uvádí všechna pole jako nepovinná – chybí v ní požadavek na povinná pole.

Důkaz z DB:

```
SELECT * FROM students WHERE id = 359;
id |first_name|last_name|email|is_eu_citizen|zip_code|number_of_failed_studies|
359|NULL      |NULL     |NULL |false        |        |                       0|
```

BUG-07 – Jedinečnost e-mailu rozlišuje velká a malá písmena

Kroky k reprodukci: 1. Existuje student s oksana.test2.jana@example.com. 2. Založit studenta s OKSANA.TEST2.JANA@EXAMPLE.COM.

Požadavek:

```
POST https://test-app.engeto.cz/students/
Authorization: Bearer <token>
Content-Type: application/json

{ "firstName": "Jana", "lastName": "Velka", "email":
"OKSANA.TEST2.JANA@EXAMPLE.COM" }
```

Očekávaný výsledek: 400, e-mail už existuje.

Skutečný výsledek: 201 Created, záznam 360 – stejná adresa je v DB dvakrát.

Důkaz z DB:

```
SELECT id, email FROM students WHERE lower(email) = 'oksana.test2.jana@example.com';
id |email                          |
357|oksana.test2.jana@example.com|
360|OKSANA.TEST2.JANA@EXAMPLE.COM|
```